



COM3021 – Data Science at Scale

Thread-level parallelism

DR YANDA MENG



Introduction

- We have presented parallelism at a system (node) level

We can also use it at other levels

Process

Instruction

Data

Thread

Thread-level parallelism is a programming model that leverages on multi-core processors

Introduction

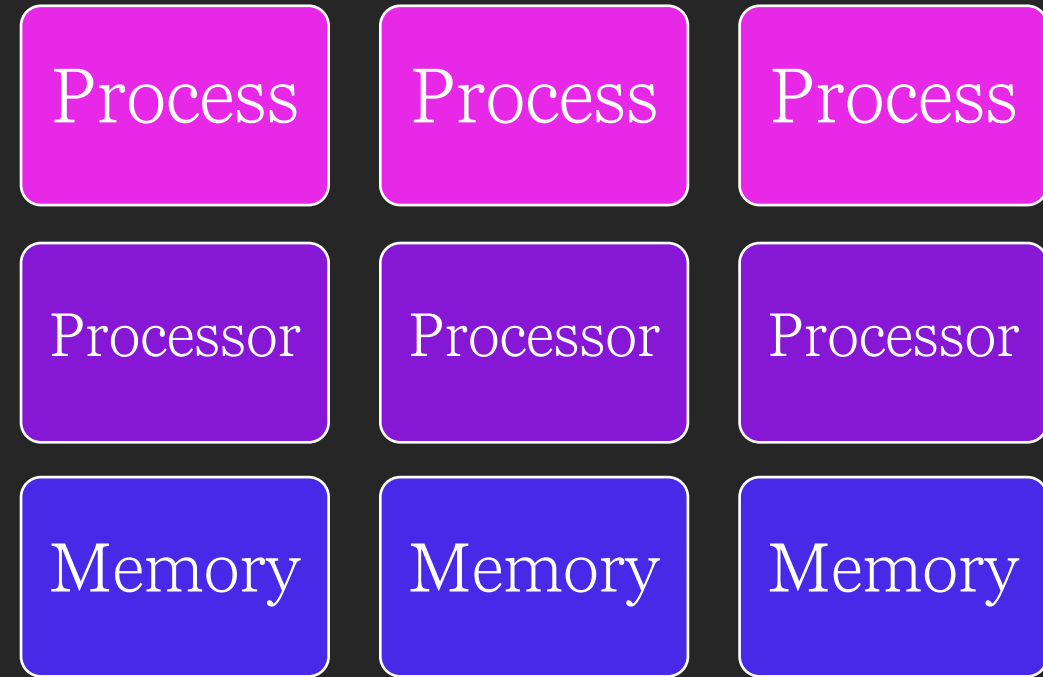
- We have presented parallelism at a system (node) level

We can also use it at other levels

Process

Thread

Spawning individual processes to achieve parallelism is expensive



Process-level parallelism

Introduction

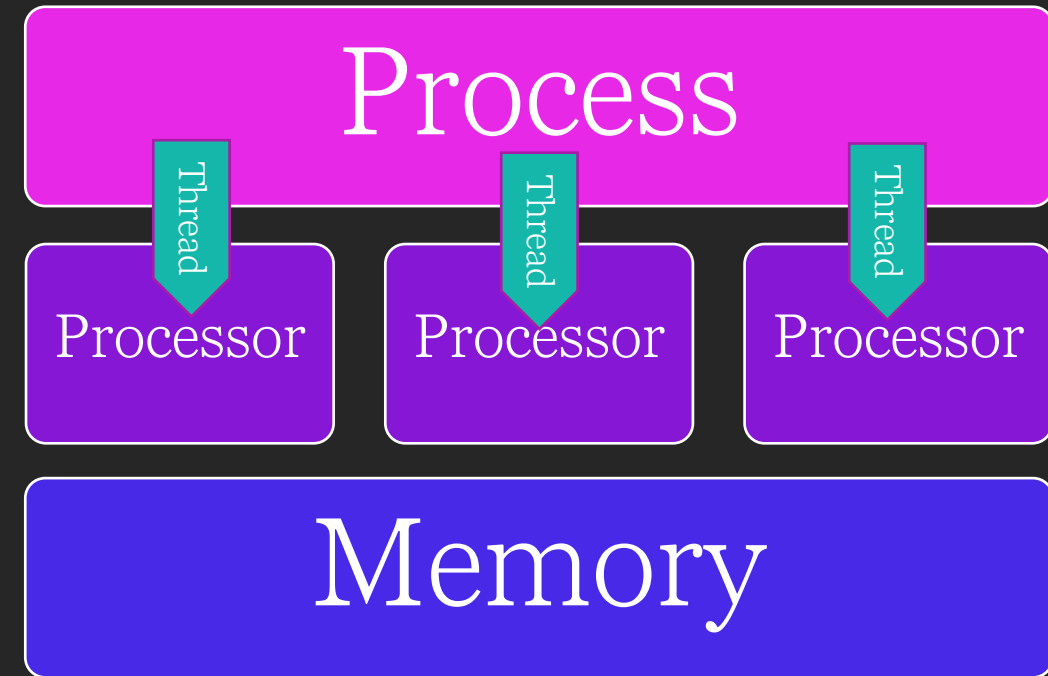
- We have presented parallelism at a system (node) level

We can also use it at other levels

Process

Thread

Spawning individual processes to achieve parallelism is expensive



Thread-level parallelism

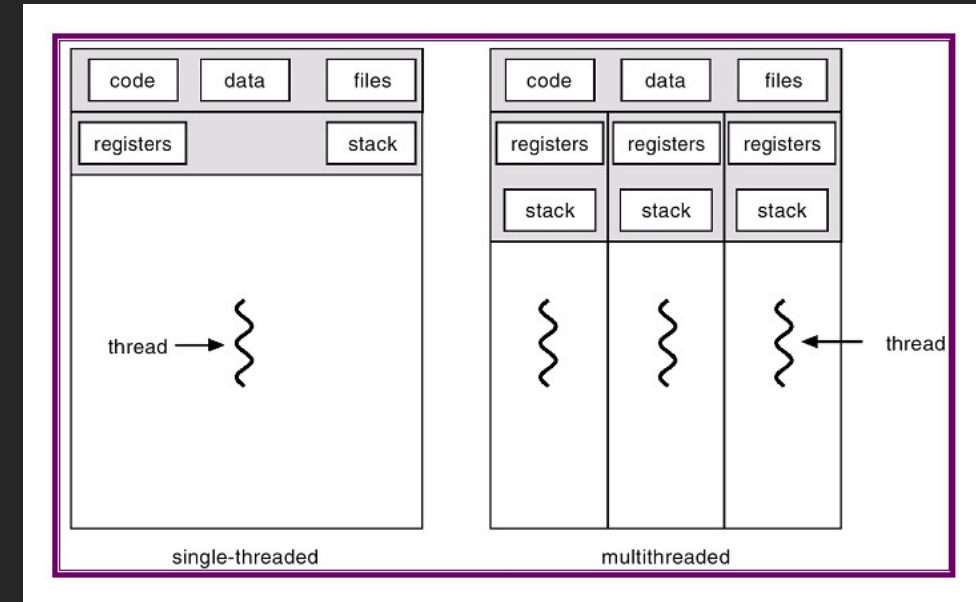
Parallel architectures

- Single instruction stream, single data stream (SISD)
uniprocessor.
 - Single instruction stream, multiple data streams (SIMD)—
 - The same instruction is executed by multiple processors using different data streams. Each processor has its own data memory (hence multiple data),
 - Multiple instruction streams, multiple data streams (MIMD)
Each processor fetches its own instructions and operates on its own data. The processors are often off-the-shelf microprocessors.
-

Thread

- Threads are semi-independent stream of execution spawned by a process
- Share memory with each other (e.g., global variables, file descriptors)

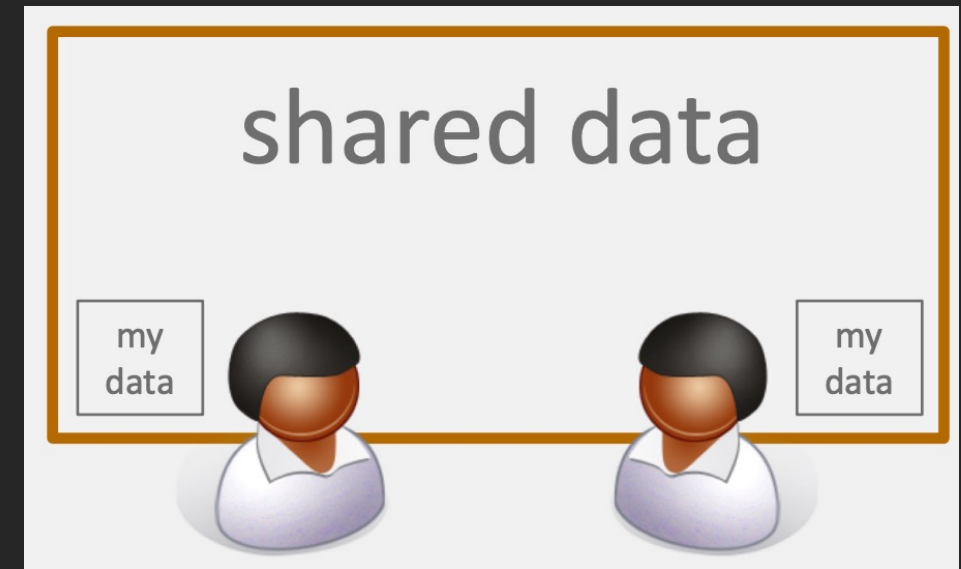
Lightweight process



Shared memory systems

- Threads can see the data of the parent process
- These threads are assigned to different physical cores on the processor
- The threads can operate on different parts of the data – this allows for parallel speed up
- Require cores to share memory

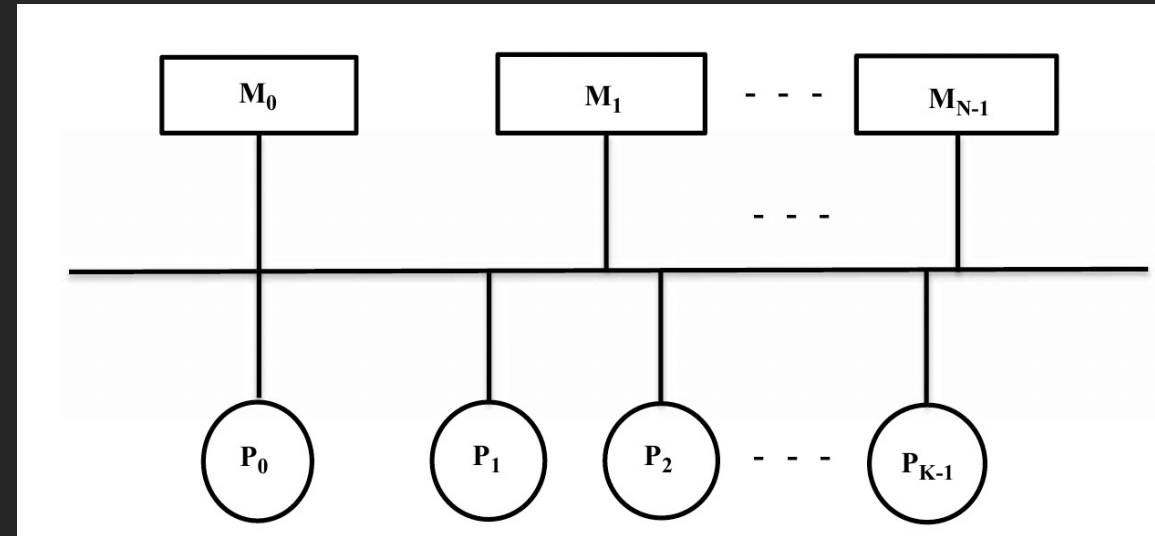
Single node



Shared memory systems

- Threads can see the data of the parent process
- These threads are assigned to different physical cores on the processor
- The threads can operate on different parts of the data – this allows for parallel speed up
- Require cores to share memory

Single node



OpenMP

- OpenMP is one of the most widely used Application Program Interface (API) for shared memory programming using threads
- Parallelism is implicit ie. a lot is abstracted from the programmer

You specify which parts of the program you want to parallelise and the compiler produces a parallel executable

OpenMP

- One parent/master thread
- Multiple worker threads
- Fork-join parallel computing

Fork

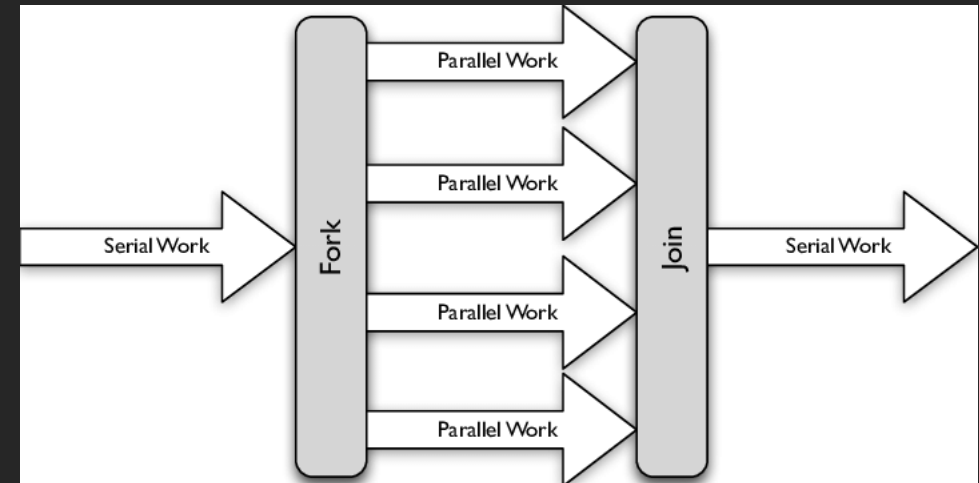
- *The parent thread spawns a number of worker threads*
- *These threads perform their computation separately*
- *Wait for the other threads to finish*

Join

- When the threads complete their computation
 - Synchronisation point before computation is continued beyond that point
-

OpenMP

- An OpenMP parallel program mostly consists of a sequence of such fork-join worker and master thread parallel segments separated by lone sequential master thread segments

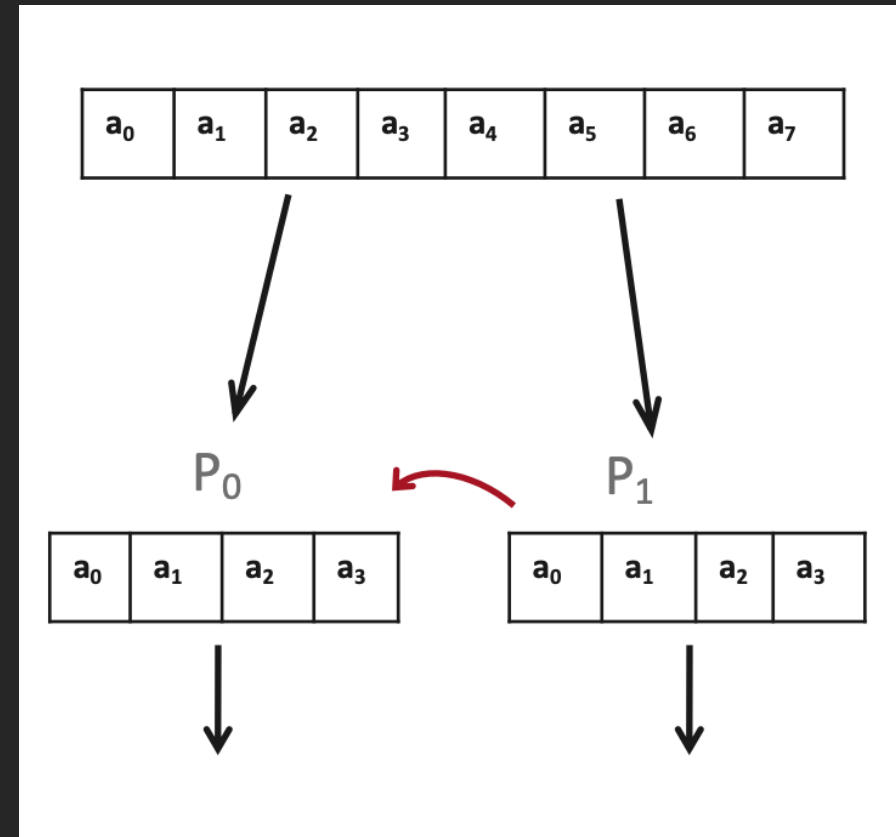


Source: Beckingsale, David. (2015). Towards Scalable Adaptive Mesh Refinement on Future Parallel Architectures.

Distributed memory parallelism

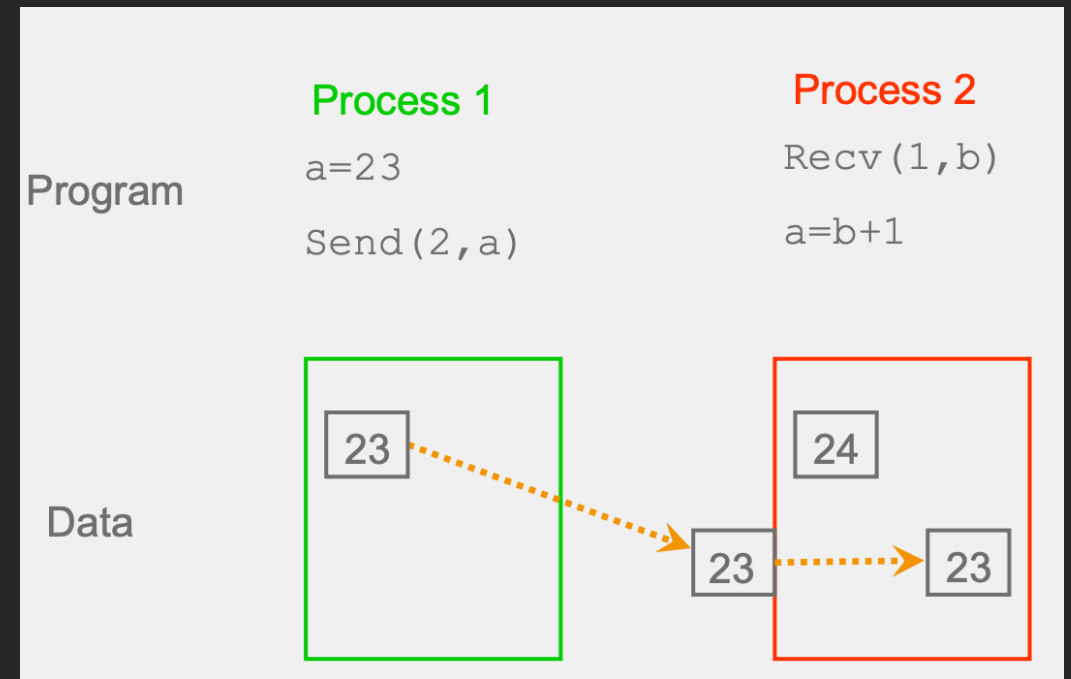
- The data is distributed amongst the processes.
- Create smaller sub-problems - parallelism
- Processes can only see their own data.
- Communication between them is done via messages.

A process can send data to another process



Process communication

- Explicit messages between processes.
- Messages are two-sided
they require a send and a receive.
- To send a message must specify the data to send and the destination process.
- The receive counterpart must match the sender process number.



Message Passing Interface (MPI)

- Standard specification of a library that contains functions which the parallel copies of a distributed memory program can use to exchange messages with each other;
 - It addresses the parallel programming model based on message passing
 - *data from the address space of one process is moved into the address space of another process via a cooperative operations inside the processes;*
 - *Topology, synchronisation and communication functionality between a set of processes across nodes*
 - Language-independent with bindings for many different languages
-

Message Passing Interface (MPI)

- Communicator
 - Objects connecting groups of processes in an MPI session
 - Give each process a unique identifier and arrange them in a topology
 - Consists of multiple MPI processes
 - A process can be associated with more than one communicator
 - The main/global communicator is called World (`MPI_COMM_WORLD`)
-

Message Passing Interface (MPI)

- Communicator
 - Several attributes, among them:
 - Context (or ID)
An identifier that differentiates one communicator from another
 - Group
The set of processes managed by that communicator
 - Size
number of processes in that communicator
 - Rank
MPI assigns an integer as an ID to each process (starting from 0).
Rank will store the Identification of each process that makes up that communicator;
-

Message Passing Interface (MPI)

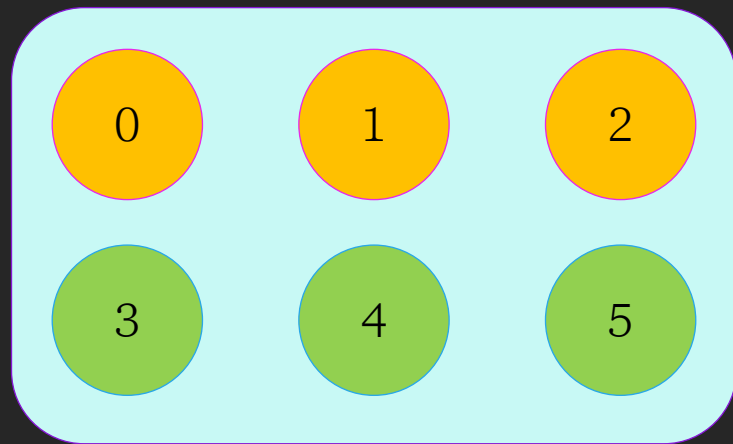
- Groups

Objects that contain the set of all processes in communicator.

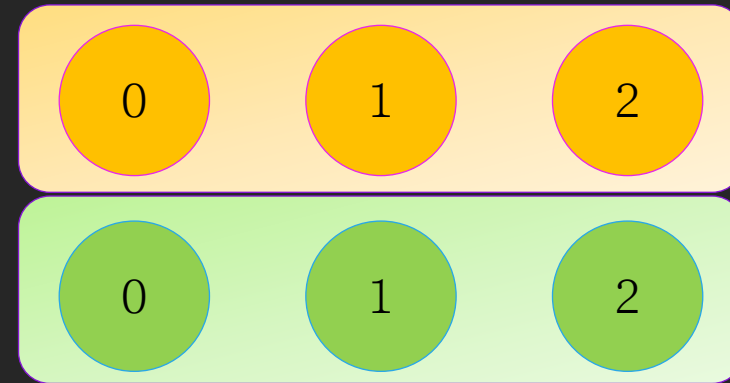
MPI_COMM_WORLD contains all processes in a particular MPI session

Message Passing Interface (MPI)

- World

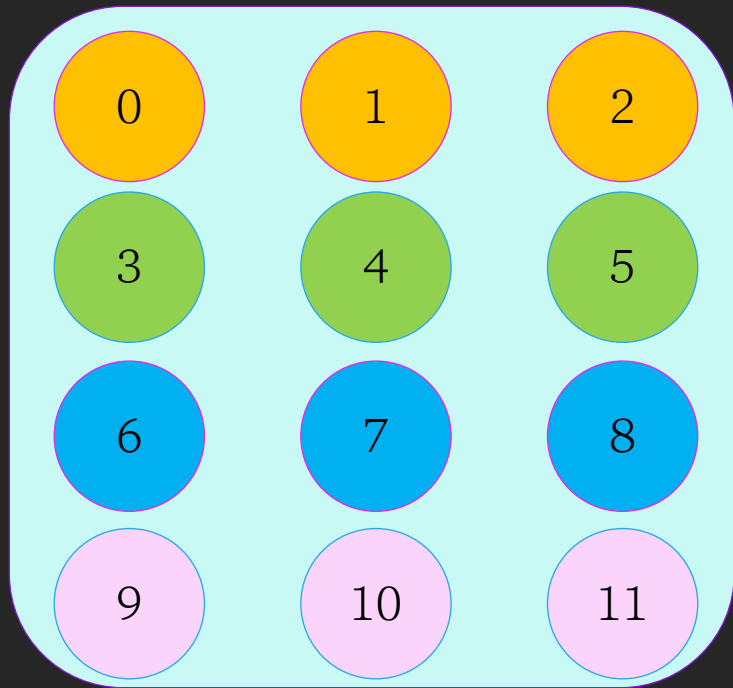


Groups

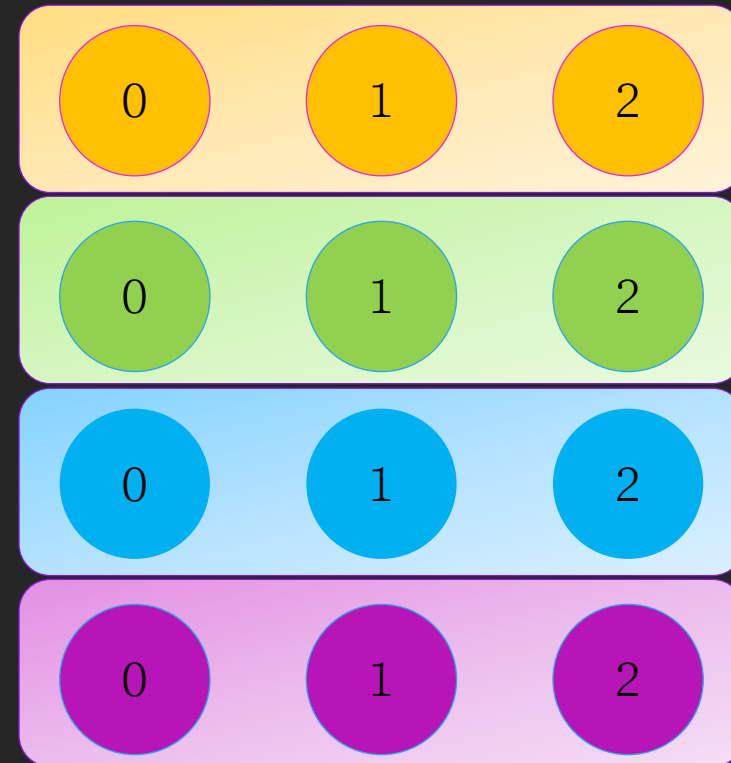


Message Passing Interface (MPI)

- World



Groups



Message Passing Interface (MPI)

- Point-to-point communications;
 - *One process (the sender) posts a message containing a piece of data targeted at a certain other process;*
 - *The other process (the receiver) expects the message;*
 - *The operation is cooperative in the sense that both the sender and the receiver must play their part in the message exchange*

```
MPI_Send(  
    void* data,  
    int count,  
    MPI_Datatype datatype,  
    int destination,  
    int tag,  
    MPI_Comm communicator)
```

```
MPI_Recv(  
    void* data,  
    int count,  
    MPI_Datatype datatype,  
    int source,  
    int tag,  
    MPI_Comm communicator,  
    MPI_Status* status)
```